TEXORA · SESSIONAL NOTES · LAB EXPERIMENT SHEET

Simple Programming for One-Dimensional and Two-Dimensional Arrays

One variable, many values: declaring arrays, walking them with loops, and the two-dimensional array as a table of rows and columns — the data structure behind Experiment 1's spreadsheet.

CSE-2106 · FUNDAMENTALS OF COMPUTING SESSIONAL · EXPERIMENT 6 · LEVEL 2 TERM 1

1. Aim

To declare, initialise, read and process one-dimensional arrays (sum, average, maximum) and two-dimensional arrays (matrix addition and printing) in C.

2. Concepts you need

- **Declaration:** `int a[5];` reserves 5 ints. **Indices run 0 to size−1** — a[0] is the first element, a[4] the last; a[5] does not exist, and C will not warn you if you touch it.
- **Initialisation:** `int a[5] = {78, 65, 82, 90, 71};` fills the array at declaration.
- **The loop partnership:** arrays are almost always processed with a for loop whose index is the subscript: `for (i = 0; i < 5; i++)`. The condition is i < size, never i <= size.
- **Partial processing:** the same skeleton finds a sum (accumulate), a maximum (keep a champion, as in Experiment 3), or a count (test each element).
- **2D arrays:** `int m[2][3];` is 2 rows × 3 columns; element m[i][j] is row i, column j. Processing needs **nested loops** — the outer loop walks rows, the inner walks columns of that row.

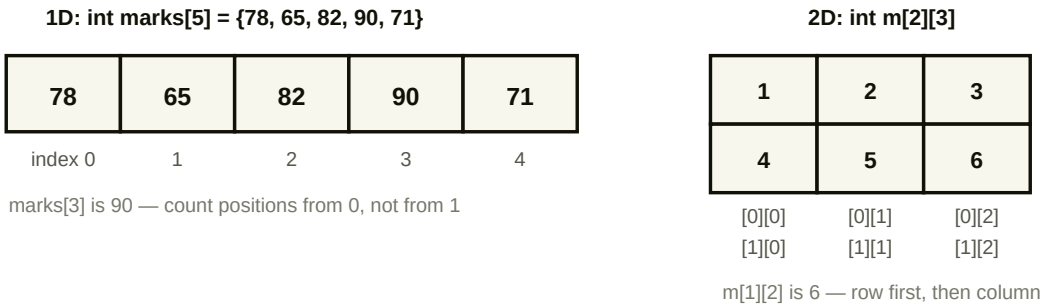


Figure 1 — Array layouts. Both worked examples below use exactly these values, so the diagram is also your trace table.

3. Lab procedure

1. Enter each program; predict the full output before running (the figure above gives you the data to trace against).
2. Change the loop condition in Example A to i <= 5 in a copy and describe what happens and why it is dangerous even when the output "looks fine".
3. For every program, run it twice with different data of your own and verify both runs by hand.

4. Worked examples

A — Sum, average and maximum of a 1D array

```
#include <stdio.h>

int main(void)
{
    int marks[5] = {78, 65, 82, 90, 71};
    int i, sum = 0, max;
    for (i = 0; i < 5; i++)
        sum = sum + marks[i];
    max = marks[0];
    for (i = 1; i < 5; i++)
    {
        if (marks[i] > max)
            max = marks[i];
    }
    printf("Sum = %d\n", sum);
    printf("Average = %.2f\n", sum / 5.0);
    printf("Maximum = %d\n", max);
    return 0;
}
```

Output:

```
Sum = 386
Average = 77.20
Maximum = 90
```

Hand check: sum runs 78 → 143 → 225 → 315 → 386 ✓; average = 386 / 5.0 = 77.20 (note the 5.0 — sum / 5 would be integer division giving 77) ✓; max walks 78 → 82 → 90 ✓. The champion starts as marks[0], not 0 — starting a maximum search at 0 silently fails if all values are negative.

B — Addition of two 2×3 matrices

```c
#include <stdio.h>

int main(void)
{
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int b[2][3] = {{7, 8, 9}, {10, 11, 12}};
    int c[2][3];
    int i, j;
    for (i = 0; i < 2; i++)
    {
        for (j = 0; j < 3; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
            printf("%d ", c[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
8 10 12
14 16 18
```

Hand check: element-wise — 1+7=8, 2+8=10, 3+9=12; 4+10=14, 5+11=16, 6+12=18. ✓ The newline is printed after the inner loop finishes a row — move it inside and the "matrix" collapses into a shamble; layout is logic.

5. Practice tasks

1. Write a program that stores {12, 7, 9, 20, 33, 46} in an array and counts how many elements are even. Give the output with a trace of the count.

2. Write a program that finds the sum of the main diagonal of the 3×3 matrix {{2, 4, 6}, {1, 3, 5}, {7, 8, 9}}. Give the output and state the index condition that picks out the diagonal.
3. In Example A, a student initialises the maximum search with max = 0. Construct a data set of five marks for which the program would report a wrong maximum, and explain the failure.

Answer key

1.
```c
#include <stdio.h>

int main(void)
{
    int a[6] = {12, 7, 9, 20, 33, 46};
    int i, count = 0;
    for (i = 0; i < 6; i++)
    {
        if (a[i] % 2 == 0)
            count++;
    }
    printf("Even numbers = %d\n", count);
    return 0;
}
```

   Trace: 12 even → 1; 7 no; 9 no; 20 even → 2; 33 no; 46 even → 3. Output: **Even numbers = 3**.

2.
```c
#include <stdio.h>

int main(void)
{
    int m[3][3] = {{2, 4, 6}, {1, 3, 5}, {7, 8, 9}};
    int i, sum = 0;
    for (i = 0; i < 3; i++)
        sum = sum + m[i][i];
    printf("Diagonal sum = %d\n", sum);
    return 0;
}
```

   The diagonal is exactly where row index equals column index (i == j). Sum = m[0][0] + m[1][1] + m[2][2] = 2 + 3 + 9 = **14**. Output: **Diagonal sum = 14**.

3. Any all-negative data set, e.g. temperature changes {−3, −7, −2, −9, −4}: the true maximum is −2, but the champion starting at 0 is never beaten, so the program reports 0 — a value not even in the array. A search champion must always start as an actual element (marks[0]), never as a convenient number.
